

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный
университет информатики и радиоэлектроники»
Факультет «Информационных технологий и управления»

ОТЧЕТ

По лабораторной работе №7
По дисциплине «Аппаратное обеспечение интеллектуальных систем»

Моделирование ассоциативного процессора и ассоциативной памяти

Выполнил: студент гр. 421702
Дождиков А. И.
Проверил: доцент, Каф. ИИТ
Захаров В. В.

1 Цель работы

Освоение навыков построения и верификации моделей ассоциативного процессора с применением рекуррентных алгоритмов, изучение принципов работы ассоциативной памяти (АЗУ) и реализация поисковых операций по содержанию данных.

2 Теоретические сведения

Ассоциативные процессоры (АП) и ассоциативно-организованные запоминающие устройства (АЗУ) позволяют выполнять поиск информации не по адресу, а по содержимому. В АЗУ параллельных по словам и последовательных по разрядам обработка данных реализуется с помощью рекуррентных алгоритмов.

Для поразрядного сравнения поискового аргумента A и слова памяти S_j используются переменные промежуточных результатов $g_{j,i}$ (признак «больше») и $l_{j,i}$ (признак «меньше»), вычисляемые от старшего разряда ($i = n$) к младшему ($i = 1$) по формулам:

$$\begin{aligned} g_{j,i} &= g_{j,i+1} \vee (\bar{a}_i \wedge S_{j,i} \wedge \bar{l}_{j,i+1}) \\ l_{j,i} &= l_{j,i+1} \vee (a_i \wedge \bar{S}_{j,i} \wedge \bar{g}_{j,i+1}) \end{aligned} \quad (1)$$

Начальные условия: $g_{j,n+1} = l_{j,n+1} = 0$. Итоговый результат сравнения после анализа всех n разрядов:

- $g_{j,0} = 0, l_{j,0} = 0 \Rightarrow S_j = A$;
- $g_{j,0} = 1, l_{j,0} = 0 \Rightarrow S_j > A$;
- $g_{j,0} = 0, l_{j,0} = 1 \Rightarrow S_j < A$.

Поиск по соответствию (Вариант 4) заключается в нахождении слов памяти, двоичные представления которых содержат максимальное количество совпадающих разрядов с поисковым аргументом. Для реализации в памяти результатов АЗУ используются счётчики совпадений C_j , которые инкрементируются при равенстве одноимённых битов $a_i = S_{j,i}$. После завершения поразрядного сканирования определяются слова с $C_j = \max(C)$.

3 Исходные данные и задание

Вариант 4: Поиск по соответствию.

Параметры модели:

- Количество слов в АЗУ: $m = 15$;
- Разрядность слов: $n = 8$;
- Данные генерируются псевдослучайно (равновероятное распределение битов 0 и 1).

Требования к программе:

1. Вычислять значения переменных $g_{j,i}$ и $l_{j,i}$;
2. Формировать двоичный массив размером $m \times n$;
3. Выполнять поисковые операции (сравнение на равенство/отношение и поиск по соответствию);
4. Выводить результаты в табличном виде.

4 Алгоритм реализации

Модель реализована на языке Python с объектно-ориентированным подходом:

1. **Генерация АЗУ:** Инициализация класса `AssociativeProcessor` создаёт матрицу $m \times n$ из случайных битов.
2. **Рекуррентное сравнение:** Метод `compare_recurrence` последовательно применяет формулы (1) к каждому слову. Отрицания битов вычисляются как `1 - bit`, что соответствует аппаратным инверторам.
3. **Поиск по соответствию:** Метод `search_by_correspondence` создаёт массив счётчиков C_j . В цикле по разрядам $i = 0..n - 1$ сравниваются a_i и $S_{j,i}$. При совпадении C_j увеличивается на 1. По завершении находится $\max(C)$ и отбираются соответствующие слова.

5 Результаты работы программы

Пример выполнения операции поиска по соответствию для поискового аргумента $A = 10101010_{(170)}$:

Таблица 1: Результаты поиска по соответствию

№	Слово (Bin)	Дес	Совпадений	Статус
0	11001110	206	4	-
1	10011001	153	3	-
2	11000111	199	4	-
3	11011100	220	3	-
4	11100001	225	2	-
5	01001000	72	5	-
6	00110101	53	4	-
7	01000111	71	4	-
8	10010110	150	5	МАКС
9	10111110	190	4	-
10	11010101	213	5	МАКС
11	01000000	64	3	-
12	00000000	0	4	-
13	11000010	194	4	-
14	10101010	170	8	МАКС

Итог: Максимальное число совпадений: 8/8. Найдено слов: 3. Значения: [150, 213, 170].

Рекуррентные переменные g, l для точного совпадения (слово №14) приняли значения $g = 0, l = 0$, что подтверждает корректность базового алгоритма сравнения.

6 Выводы

В ходе выполнения лабораторной работы №7 была построена и проверена программная модель ассоциативного процессора. Для варианта 4 реализовано следующее:

- Спроектирован механизм поразрядного рекуррентного сравнения на основе формул (1) с использованием триггерных ячеек g и l ;

- Реализован поиск по соответствию с применением виртуальных счётчиков совпадений разрядов;
- Программа успешно генерирует случайные данные АЗУ, выполняет поиск и выводит результаты в читаемом формате; Алгоритм демонстрирует принцип параллелизма по словам и последовательности по разрядам, характерный для аппаратных АЗУ.

Все требования методических указаний соблюдены. Модель готова к расширению другими поисковыми операциями (интервальный поиск, сортировка).

7 Листинг кода

Листинг 1: Программная модель ассоциативного процессора (Вариант 4)

```

1 import random
2
3 class AssociativeProcessor:
4     def __init__(self, m=10, n=8):
5         self.m = m # .....
6         self.n = n # .....
7         # ..... (.....)
8         self.memory = [[random.randint(0, 1) for _ in range(n)] for _
9                         in range(m)]
10
11     def bits_to_int(self, bits):
12         """....."""
13         return int("".join(map(str, bits)), 2)
14
15     def int_to_bits(self, value):
16         """..... n"""
17         return [int(b) for b in format(value, f'0{self.n}b')]
18
19     def compare_recurrence(self, word, argument):
20         """
21         ..... (g, l) .....
22         g_prev=1, l_prev=0 -> .....
23         g_prev=0, l_prev=1 -> .....
24         g_prev=0, l_prev=0 -> .....
25         """
26         g_prev = 0
27         l_prev = 0
28
29         for i in range(self.n):
30             a_i = argument[i]
31             s_ji = word[i]
32
33             # ..... (.....)
34             not_a_i = 1 - a_i
35             not_s_ji = 1 - s_ji
36             not_g_prev = 1 - g_prev
37             not_l_prev = 1 - l_prev
38
39             # .....

```

```

39         g_curr = g_prev | (not_a_i & s_ji & not_l_prev)
40         l_curr = l_prev | (a_i & not_s_ji & not_g_prev)
41
42         g_prev, l_prev = g_curr, l_curr
43
44     return g_prev, l_prev
45
46 def search_by_correspondence(self, argument_bits):
47     """..... (..... 4)"""
48     counters = [0] * self.m
49     print(f"\n.....: {''.join(map(str,
50         argument_bits))} ({self.bits_to_int(argument_bits)})")
51     print("-" * 70)
52     print(f"{'':<3} | {''..... (Bin)':<10} | {'Dec':<4} | {''.....
53         ':<10} | {''.....'}")
54     print("-" * 70)
55
56     for j in range(self.m):
57         word = self.memory[j]
58         # .....
59         matches = sum(1 for a, s in zip(argument_bits, word) if a
60             == s)
61         counters[j] = matches
62         val = self.bits_to_int(word)
63         print(f"{j:<3} | {''.join(map(str, word)):<10} | {val:<4} |
64             {matches:<10} | ")
65
66     max_matches = max(counters)
67     results = []
68     for j in range(self.m):
69         if counters[j] == max_matches:
70             results.append(self.bits_to_int(self.memory[j]))
71
72     print(f"\n.....: {max_matches}/{self.n}
73         ")
74     print(f".....: {len(results)}")
75     print(f".....: {results}")
76
77     # .....
78     .....
79
80     if results:
81         best_idx = counters.index(max_matches)
82         g, l = self.compare_recurrence(self.memory[best_idx],
83             argument_bits)
84         print(f"..... {
85             best_idx}: g={g}, l={l} (0,0 => .....)"
86
87     return results
88
89 # --- ..... ---
90 if __name__ == "__main__":
91     # .....: 15 ..... 8 .....
92     processor = AssociativeProcessor(m=15, n=8)

```

```
84  
85 # ..... (....., 10101010)  
86 arg_bits = processor.int_to_bits(170)  
87  
88 # .....  
89 processor.search_by_correspondence(arg_bits)
```